Day 2 — First Python Program & Basic Syntax

Detailed Seminar Notes | print() • Syntax • Comments • Errors • Rules • Examples • Viva

Day 2 is about writing your **first real Python programs** and understanding the **basic syntax rules** that every beginner must know. In this day, you will learn how Python statements are written, how the **print()** function works, how to display text and numbers, how to write comments, and what common syntax mistakes beginners make.

1) What do we mean by a Python program?

A **Python program** is a set of instructions written in Python that tells the computer what to do. Even a single line like **print("Hello")** is a Python program because it gives the computer a clear instruction: display the word **Hello** on the screen.

2) Structure of a simple Python program

At the beginning level, a Python program may contain:

- **Statements** — instructions written in Python.
- **Functions** — ready-made blocks like **print()** used to perform tasks.
- **Values** — text, numbers, and other data used inside statements.
- **Comments** — notes written by the programmer for explanation.

3) The print() function

The **print()** function is one of the most important beginner functions in Python. It is used to **display output** on the screen.

General syntax:

```
print(value)
```

The value inside **print()** can be:

- A **string** like **"Hello"**
- A **number** like **100**
- An **expression** like **10 + 20**
- Multiple values separated by commas

Example 1: Print text

```
print("Hello, World!")
```

Output: Hello, World!

Example 2: Print a number

```
print(100)
```

Output: 100

Example 3: Print the result of an expression

```
print(10 + 20)
```

Output: 30

Example 4: Print multiple values

```
print("Age =", 19)
```

Output: Age = 19

4) Understanding print() properly

Part	Meaning
print	Built-in Python function used to display output
()	Parentheses used to pass values to the function
"Hello"	A string value written inside quotes
100	A numeric value
10 + 20	An arithmetic expression whose result will be printed

5) Printing strings and numbers

In Python, text values are called **strings**. Strings must be enclosed in quotes. Numbers are written without quotes.

Code	Type of value	Output
print("Python")	String	Python
print(50)	Number	50
print("50")	String	50

Notice that **50** and **"50"** may look similar on the screen, but internally they are different. One is a **number** and the other is a **string**.

6) Rules for writing strings

- Strings must be enclosed in **single quotes** or **double quotes**.
- The opening and closing quotes must match.
- If you forget quotes around text, Python will show an error.
- Text like **Hello** should be written as **"Hello"**.

Correct examples

```
print("Python")
print('Python')
```

Wrong example

```
print(Python)
```

This is wrong because **Python** is written without quotes, so Python will think it is a variable name and raise an error.

7) Basic syntax rules in Python

- Python is **case sensitive**. **print** and **Print** are not the same.
- Function names must be written correctly. Use **print()**, not **Print()**.

- Parentheses must open and close properly.
- Strings must be written inside matching quotes.
- Each statement should be written clearly and correctly.
- Indentation becomes important in later topics like conditions, loops, and functions.

8) Comments in Python

A **comment** is a note written in the code to explain something. Python ignores comments during execution.

Syntax of single-line comment:

```
# This is a comment
```

Example

```
# This program prints a welcome message
print("Welcome to Python")
```

Comments help other people and your future self understand what the code is doing.

9) More examples of print()

Program 1: Print your name

```
print("Adi")
```

Program 2: Print your college and branch

```
print("B.Tech CSE")
```

Program 3: Print two lines

```
print("Hello")
print("Python")
```

Program 4: Print text and number together

```
print("Age =", 19)
print("Year =", 2026)
```

10) Printing multiple values with commas

In Python, we can print more than one value in a single **print()** statement by separating them with commas.

```
print("Name:", "Adi")
print("Marks:", 95)
print("A", "B", "C")
```

Output:

```
Name: Adi
Marks: 95
A B C
```

By default, Python puts a **space** between values when commas are used inside **print()**.

11) sep and end in print()

The **print()** function has useful optional arguments like **sep** and **end**.

a) sep

sep means separator. It changes the symbol placed between multiple printed values.

```
print("2026", "07", "08", sep="-")
```

Output: 2026-07-08

b) end

By default, **print()** moves to a new line after printing. The **end** argument changes what is printed at the end.

```
print("Hello", end=" ")
print("Python")
```

Output: Hello Python

12) Escape sequences used with print()

Escape sequences are special characters that begin with a backslash **** and are used inside strings.

Escape sequence	Meaning	Example
\n	New line	print("Hello\nPython")
\t	Tab space	print("A\tB")
\"	Double quote inside string	print("He said \"Hi\"")

Examples

```
print("Hello\nPython")
print("A\tB")
print("He said \"Hi\"")
```

13) Common syntax errors beginners make

Wrong code	Why it is wrong	Correct form
Print("Hello")	Python is case sensitive. Function name is wrong	print("Hello")
print("Hello)	Closing quote is missing.	print("Hello")
print("Hello"	Closing parenthesis is missing.	print("Hello")
print(Hello)	Text is written without quotes.	print("Hello")

14) Runtime vs syntax understanding at beginner level

A **syntax error** happens when the program breaks the rules of writing Python code. For example, missing quotes or wrong parentheses.

A **runtime error** happens while the program is running. At this stage, the most important thing is to write syntax correctly so the program can execute properly.

15) Practice programs for Day 2

Program 1: Print a welcome message

```
print("Welcome to Python Programming")
```

Program 2: Print your details

```
print("Name: Adi")
print("Course: B.Tech CSE")
print("Year: 3rd Year")
```

Program 3: Print two numbers and their sum

```
print(10)
print(20)
print(10 + 20)
```

Program 4: Use sep

```
print("2026", "07", "08", sep="/")
```

Program 5: Use end

```
print("Python", end=" ")
print("Programming")
```

Program 6: Use escape sequences

```
print("Line1\nLine2")
print("A\tB\tC")
```

16) How to explain Day 2 in a seminar

- Start by telling that a Python program is a set of instructions.
- Explain the **print()** function first because it is the easiest and most basic function.
- Show examples of printing text, numbers, and expressions.
- Explain that strings need quotes but numbers do not.
- Tell the syntax rules: case sensitivity, parentheses, and matching quotes.
- Explain comments and why they are useful.
- Demonstrate **sep** and **end** if you want to show slightly advanced print usage.
- Finish with common errors and practice programs.

17) Viva / seminar questions for Day 2

- What is a Python program?
- What is the use of print()?
- Can print() display both strings and numbers?
- Why must strings be written inside quotes?
- What is the difference between **print(50)** and **print("50")**?
- What is a comment in Python?
- Why is **Print()** wrong in Python?
- What is the use of **sep** in print()?

- What is the use of **end** in print()?
- Give examples of common syntax errors in Python.

18) Short seminar summary

Day 2 introduces the student to writing actual Python statements. The most important concept is the **print()** function, which is used to display output. Along with that, students learn how strings and numbers are written, how comments work, what basic syntax rules must be followed, and how common beginner errors can be avoided.

19) Quick revision box

Topic	Key point
Python program	A set of instructions written in Python
print()	Used to display output on the screen
String	Text written inside single or double quotes
Number	Numeric value written without quotes
Comment	Written using # and ignored by Python
sep	Changes separator between printed values
end	Changes what is printed at the end of print()